Applied Machine Learning for Environmental Sciences

A hands-on workshop using iMESc — Participant handbook

Instructors: Prof. Gustavo Fonseca (Instituto do Mar, UNIFESP, Brazil) and Prof. Danilo C. Vieira (Instituto Oceanográfico, USP, Brazil)

Software: iMESc — interactive Machine Learning app for Environmental Sciences (Vieira & Fonseca)

Host: Ghent University Doctoral School — specialist course

Help pages: https://danilocvieira.github.io/iMESc_help/

All datasets used in Examples 1–3 are synthetic. They were generated to resemble real environmental data and contain a known, hidden structure, so that you can check whether the methods recover it.

Contents

Contents 2
Getting started 4
What you will do 4
Installing and starting iMESc 4
Key iMESc concepts 4
Example 1 — Finding groups along an estuarine gradient 6
Context 6
Data 6
Step 1 — Create the Datalist 6
Step 2 — Explore before modelling 6
Step 3 — Handle missing values 7
Step 4 — Transform, scale and explore a bit more 7
Step 5 — Train a Self-Organizing Map 7
Step 6 — Cluster the SOM codebook 8
Step 7 — Map and interpret the groups 8
Step 8 — Test the robustness of your answer 8
Example 2 — Predicting benthic richness and quality with spatial validation 9
Context 9
Data 9
Step 1 — Create the Datalists 10
Step 2 — Explore 10
Step 3 — Prepare the data 10
Step 4 — Create the spatial block CV scheme 10
Step 5 — Regression: predict Richness 10
Step 6 — Classification: predict Quality_class 11
Step 7 — Transfer to the new area 11
Example 3 — Forecasting coastal chlorophyll-a 12
Context 12
Data 12
The three model levels 12
Step 1 — Load and look at the series 13
Step 2 — Create the targets with lead 13
Step 3 — Create the 7-day means (level M2) 13
Step 4 — Create the M0 Datalist 13
Step 5 — Remove NA rows and align all Datalists 13
Step 6 — Set up training (the same for all six models) 14
Step 7 — Train the six models 14
Step 8 — The persistence baseline 14
Step 9 — Inspect the best model 14

Applying the workflow to your own data 16
Glossary of metrics 17

Getting started

What you will do

The workshop is built around three worked examples. Each one asks a scientific question, gives you a dataset and walks you through a complete machine-learning workflow in iMESc.

Example	ML task	Main skill
1. Estuarine gradient	Unsupervised (SOM + HC)	Pre-processing, choosing variables, finding and mapping groups
2. Shelf benthos	Supervised regression + classification	Spatial cross-validation, transfer to a new area, imbalanced classes
3. Coastal chlorophyll	Supervised forecasting (time series)	Building temporal features in iMESc, comparing model levels and horizons, avoiding temporal leakage

At the end you will apply the same workflow to your own data.

Installing and starting iMESc

1. Install a recent version of **R** and **RStudio**.
2. In RStudio, install Shiny: `install.packages("shiny")`.
3. Start iMESc with the command below. The first launch installs all dependencies and can take several minutes; later launches are faster.

```
shiny::runGitHub('imesc', 'danilocvieira')
```

> **Tip**
>
> iMESc needs a screen resolution of at least 1377 × 768 pixels. If panels overlap, maximise the browser window or zoom out.

Key iMESc concepts

Datalist. Everything in iMESc is stored in Datalists. A Datalist has one required attribute and several optional ones:

Attribute	Required?	Content
Numeric-Attribute	Yes	Continuous or discrete variables (predictors, or a numeric target)
Factor-Attribute	No	Categorical variables: groups, seasons, classes, partitions
Coords-Attribute	No	ID, Longitude, Latitude (decimal degrees), used by the Spatial tools
Temporal-Attribute	No	ID, Temporal columns associated with the observations
Shapes	No	Base, layer and extra shapes for maps
Models	No	Trained SOM and supervised models are saved inside the X Datalist

File rules for uploading CSV or XLSX files:

- The first column holds a **unique ID**; the first row holds the variable names.
- Variable names must be unique, have no spaces or special symbols, and must not start with a number.
- Use a dot as decimal separator, and write missing values as `NA`.

- All files of one Datalist must use the same IDs.

The four recurring steps of every workflow:

1) **Create Datalists** according to the learning approach:
   a) Unsupervised learning: X only (numeric predictors).
   b) Supervised learning:
      i) Classification: X (numeric predictors) and Y (factor response).
      ii) Regression: X (numeric predictors) and Y (numeric response).
2) **Pre-process**: impute missing values, transform, and partition Y into training and test sets. No partition is provided in the files: you create every partition yourself in iMESc (Data partition, a spatial CV scheme, or the `Year` factor).
3) **Save** changes and models. A flashing blue disc button means there is something to save.
4) **Create a savepoint** (Pre-processing tools → Savepoint) regularly. It downloads your whole workspace as an .rds file that you can reload later.

> **Tip**
>
> Save a savepoint at the end of every example. If iMESc crashes or you close the browser, you can reload it instead of starting again.

Example 1 — Finding groups along an estuarine gradient

Context

Estuaries are among the most productive and most impacted coastal systems. Along the few tens of kilometres between the river and the sea, salinity, turbidity, nutrients and oxygen change steeply. Environmental agencies divide estuaries into zones (for example tidal freshwater, oligohaline, mesohaline, polyhaline) to set monitoring and management targets. These zones are usually drawn from salinity thresholds alone. However, a turbidity maximum, sediment type, the season, or a sewage or industrial outfall can all create water bodies with their own identity. Machine learning lets us ask the data directly how many distinct environments there are and where they are, without imposing thresholds beforehand.

Scientific question

How many distinct environmental zones exist along this estuary, what defines each one, and are they stable between seasons?

- Is the estuary better described as discrete zones or as a continuous gradient?
- Can unsupervised learning detect a local anthropogenic impact (an outfall) without being told it exists?
- How do pre-processing choices (transformation, scaling, variable selection) change the answer?

Data

240 water and sediment samples: 120 stations, each sampled in summer and in winter.

File	Load as	Content
`ex1_numeric.csv`	Numeric-Attribute	16 variables: Salinity, Conductivity, Temperature, Turbidity, SPM, DO, DO_sat, NO3, NH4, PO4, Chla, Mud, TOC, Depth, Wind_speed, Tide_height
`ex1_factors.csv`	Factor-Attribute	Season (Summer/Winter), Campaign
`ex1_coords.csv`	Coords-Attribute	Longitude, Latitude

Tip

The dataset contains missing values, skewed variables, variables that measure the same thing, and variables with no information. Finding them is part of the exercise.

Step 1 — Create the Datalist

1. Pre-processing tools → **Create Datalist** → upload `ex1_numeric.csv` as Numeric-Attribute, `ex1_factors.csv` as Factor-Attribute and `ex1_coords.csv` as Coords-Attribute.
2. Name it `estuary_raw`.
3. Open **Data Bank** and check the Numeric-Attribute and Factor-Attribute tables.

Checkpoint

- The Datalist has 240 observations and 16 numeric variables.

Step 2 — Explore before modelling

1. Descriptive tools → **Summary**: note which variables have NAs and how many.
2. Descriptive tools → **Ridges** and **Boxplot**: which variables are strongly right-skewed?

Think about it

- Are the NAs scattered at random, or concentrated in one campaign or one variable? Why does that matter for imputation?
- Two variables with r ≈ 1 carry the same information. What happens to a distance-based method if both are kept?
- Is there any variable that does not correlate with anything?

Step 3 — Handle missing values

1. Pre-processing tools → **Data imputation** → method **Knn**. Save as `estuary_knn`.
2. Repeat with **MedianImpute** and **rf** (save as new Datalists).
3. Compare the imputed `Mud` values between methods (Boxplot).

Think about it

- Mud and TOC are missing mostly at sandy stations, where grab samplers often fail. Will imputation over- or underestimate Mud there?

Step 4 — Transform, scale and explore a bit more

1. Pre-processing tools → **Transformations** → apply **Log10(x+1)** (or **Log10**) to the skewed variables found in Step 2.
2. Tick **Scale** and **Center** so that all variables have mean 0 and SD 1.
3. Optionally remove redundant and uninformative variables (Edit Datalist Columns). Keep a copy of the full version for comparison.
4. Descriptive tools → **Correlation plot** (Spearman): which pairs have $|r| > 0.9$?
5. Descriptive tools → **PCA** on the raw data, coloured by `Season`. What drives PC1 and PC2?
6. Save as `estuary_ready`.

Tip

Scaling is essential for SOM, k-means and hierarchical clustering: they use distances, so a variable measured in large units (e.g. NO3 in µM) would otherwise dominate.

Step 5 — Train a Self-Organizing Map

1. Unsupervised algorithms → **Self-Organizing Maps** → **Training**. Select `estuary_ready`.
2. Suggested grid: about 5 × √n nodes ≈ 75 → **xdim = 9, ydim = 8, hexagonal** topology; `rlen` = 500; fix the **seed** (e.g. 1).
3. Train and save the model.
4. **Results** → check the training progress (Changes), the quantization error, topographic error and neuron utilisation.
5. Inspect the pie plot: Which variables are the most important in each region of the map?

Checkpoint

- The training curve has flattened by the end of training.
- Few or no empty neurons; topographic error is low (< 0.1).

Step 6 — Cluster the SOM codebook

1. Unsupervised algorithms → **Hierarchical clustering** → target: **SOM-codebook**, method **ward.D2**, Euclidean distance.
2. Look at the **Scree plot** / **Codebook screeplot**. Do not stop at the first elbow: cut the dendrogram at several values of k (e.g. 8 to 10) and, for each, ask whether the new split has a clear environmental meaning. Choose the k where splits stop being interpretable.
3. **Cut Dendogram** at k and view **Codebook clusters**. Save the clusters to the Factor-Attribute (flashing blue button).
4. Optional: run **K-means** on the codebook with the same k and compare.

Step 7 — Map and interpret the groups

1. Spatial tools → **Circles**: colour the stations by the saved cluster factor; split by Season.
2. Descriptive tools → **Boxplot** of the original variables by cluster.
3. Write a one-line "identity" for each cluster (e.g. "high NH4, low DO, warm → impacted").

Step 8 — Test the robustness of your answer

1. Repeat Steps 5–7 **without Temperature**, or for each season separately.
2. Repeat without log-transformation and without scaling.
3. Compare the cluster maps. We will then reveal the "true" zones.

Think about it

- Did your clustering separate places, seasons, or both? Do all environments change between summer and winter, or is there one that keeps its identity?
- Which zones are always recovered, and which ones blur into each other? Is that a failure of the method or a property of the estuary?
- Which pre-processing choice changed the result the most?

Example 2 — Predicting benthic richness and quality with spatial validation

Context

Offshore oil and gas activities are monitored with benthic surveys: sediment samples are collected around platforms and across the shelf, and the macrofauna is identified. The work is slow and expensive, so there is strong interest in models that predict biological indicators (species richness, or an ecological quality class) from cheaper environmental measurements such as depth, grain size, organic carbon and drilling tracers (barium, hydrocarbons). Such models are used to map impact and to fill gaps. But samples close to each other are similar (spatial autocorrelation), and stations are often clustered around platforms. A model that looks excellent under ordinary cross-validation may therefore fail in a new area. The honest test of a predictive model is how well it predicts where it has not seen data.

Scientific question

Can benthic species richness and ecological quality status be predicted from environmental and contamination variables, and how well does such a model transfer to an unsampled area?

- Which environmental and impact variables drive richness and quality?
- How much does random cross-validation overestimate performance compared with spatial (block) validation?
- Where does the model fail when applied to a new, deeper area, and why?

Data

Training area: 400 stations (325 on a grid, 75 clustered around three platforms). **Transfer area**: 100 stations, about 100 km to the north-east and deeper, used only for the final test.

The files are complete (no missing values), so you can go straight from exploration to modelling. Training/validation partitions are not provided as files: you create them inside iMESc.

File	Datalist	Content
`ex2_train_X_numeric.csv`	`shelf_X` — Numeric	15 predictors: Depth, Dist_coast, Dist_platform, Bottom_temp, Bottom_sal, Bottom_DO, Mud, Sand, Grain_phi, TOC, CaCO3, Chla_sat, Ba, THC, Wave_exposure
`ex2_train_Y_numeric.csv`	`shelf_Y` — Numeric	Richness (number of species per sample)
`ex2_train_factors.csv`	Factor of `shelf_X` and `shelf_Y`	Quality_class (Good / Moderate / Poor), Sampling_design (Grid / Platform)
`ex2_train_coords.csv`	Coords of `shelf_X` and `shelf_Y`	Longitude, Latitude — used for maps and for the spatial blocks
`ex2_transfer_X_numeric.csv`	`transfer_X` — Numeric	The same 15 predictors, for the 100 transfer stations
`ex2_transfer_Y_numeric.csv`	`transfer_Y` — Numeric	Observed Richness in the transfer area (for the final check only)
`ex2_transfer_factors.csv`	Factor of `transfer_X` and `transfer_Y`	Quality_class, Sampling_design
`ex2_transfer_coords.csv`	Coords of `transfer_X` and `transfer_Y`	Longitude, Latitude

Step 1 — Create the Datalists

1. Create `shelf_X` (X_numeric + factors + coords) and `shelf_Y` (Y_numeric + factors + coords) from the **train** files.
2. Create `transfer_X` and `transfer_Y` in the same way from the **transfer** files. Do not look at them yet.

Step 2 — Explore

1. **Correlation plot** of `shelf_X`: find the groups of strongly correlated predictors.
2. Spatial tools → **Circles**: map Richness and Quality_class. Where are the Poor stations?
3. Cross-tabulate Quality_class by Sampling_design (Summary → Factor-Attribute).

> **Think about it**
>
> - How many Poor stations are there? What does a model that always predicts "Good" achieve in accuracy?

Step 3 — Prepare the data

1. Check with Descriptive tools → **Summary** that the Datalists are complete: the Example 2 files have **no missing values**, so no imputation is needed.
2. No partition is needed: all 400 training stations go into cross-validation, and the **transfer area is the independent test**. In the model setup, keep **Partition = None**.

Step 4 — Create the spatial block CV scheme

1. Supervised algorithms → **Random Forest** → **Model setup**: model type **Regression**, Y Datalist = `shelf_Y` (Richness), X Datalist = `shelf_X`, Partition = None.
2. In **1. Training** → **Validation**, set Method = **Spatial block CV** and click **Create New Spatial CV scheme**.
3. In the **Size Evaluation Tool** tab keep **Nº Folds = 5**, Repeats = 1, **Hexagonal spatial blocks** ticked and Selection = random; set a **seed** and click **Run size evaluation**. The tool suggests a block size (in metres); inspect the fold map and click **Use this scheme**.
4. Write down the block size. Check the folds with **See Details**: in each fold the test set is made of whole hexagonal blocks, and every station is tested once.
5. To choose the block size yourself, use the **Create Spatial Folds** tab (Size in metres, seed) and click **Create spatial CV scheme**.

> **Tip**
>
> Each hexagonal block is a compact spatial region. In every fold whole blocks are held out, which mimics predicting into an area the model has not seen.
>
> The scheme is cleared whenever the X or Y Datalist, the observations or the partition change. Create it after choosing X and Y, and again (same seed and size) for each new model.

Step 5 — Regression: predict Richness

1. **Model A — random CV:** in **1. Training** → Validation, Method = **Repeated Cross-Validation**, 5 folds, 3 repeats. Train and save.
2. **Model B — spatial CV:** Method = **Spatial block CV** with the scheme from Step 4. Train and save.
3. **2. Results**: record the cross-validated R^2, RMSE and the variable importance of both models.
4. Optional (B'): train a second algorithm (e.g. **gbm** or **svmRadial**) with the same spatial scheme and compare with **Compare Models**.

Model	Validation	R^2 (CV)	RMSE	R^2 transfer (Step 7)	Top 3 variables
A	Repeated CV (random folds)				
B	Spatial block CV, ___ m blocks				
B'	Spatial block CV, 2nd algorithm				

Step 6 — Classification: predict Quality_class

1. Same procedure with model type **Classification** and Y = `Quality_class` from the Factor-Attribute. Train Model A (random CV) and Model B (spatial block CV; create the scheme again with the same seed and block size).
2. In **2. Results**, inspect the **confusion matrix**, overall accuracy and **Kappa**.
3. Compute the recall of each class (correct / observed in that class). Which class is the hardest?

Think about it

- Why is Kappa a better summary than accuracy when one class is rare?
- Which predictors separate Poor from Good? Are they "natural" or "impact" variables?

Step 7 — Transfer to the new area

1. In **3. Predict**, apply Models A and B to `transfer_X` and compare with the observed values in `transfer_Y` (Richness) and in the Factor-Attribute (Quality_class).
2. Plot the prediction error against Depth (export the predictions or use the Results plots).
3. Retrain without `Wave_exposure` and without one variable of each collinear group. Does transfer performance change?

Think about it

- How does R^2 in the transfer area compare with R^2 from cross-validation? Which was the better predictor of transfer performance: random or spatial CV?
- Is the transfer error random, or systematic (always too high or too low) in part of the depth range? What could make the same environmental conditions support different richness in a new region?
- What happens to the predictions for stations deeper than any training station?
- For Quality_class: compare accuracy, Kappa and the confusion matrix in the transfer area with cross-validation. Are the misclassifications random, or always in the same direction (too optimistic or too pessimistic)?

Example 3 — Forecasting coastal chlorophyll-a

Context

Phytoplankton blooms on productive coastal shelves are fuelled by two nutrient sources: coastal upwelling, when persistent alongshore winds push surface water offshore and bring cold, nutrient-rich water to the surface, and river discharge after heavy rain. Both act with a delay: the bloom appears days after the wind or the rain. Satellite and buoy records of chlorophyll-a are now long enough to train forecasting models, and short-term forecasts support aquaculture management, harmful-bloom early warning and water-quality monitoring. The difficulty is that raw daily data do not "know" about delays and accumulation. The ecological knowledge has to be translated into **temporal features**, and the model has to be tested on a future it has not seen.

In this example you will build, entirely in iMESc, **three models of increasing information** and test each one at **two forecast horizons** (3 and 7 days ahead).

> **Scientific question**
>
> **Can we forecast coastal chlorophyll-a 3 and 7 days ahead, and how much does each extra level of information — the chlorophyll itself, today's environment, and the environment of the past week — improve the forecast?**
>
> - How much better than simple persistence ("in 7 days = today") can a model do?
> - Does the value of the extra information change with the horizon?
> - Which drivers carry the predictive information, and does that match the delays described above?

Data

Daily series from 1 January 2020 to 31 December 2024 (1827 days) at one coastal station (southern hemisphere: summer = December–February). There are no missing values.

Variable	Unit	Description
Chla	mg m^{-3}	Surface chlorophyll-a — the variable to forecast
Wind_along	m s^{-1}	Alongshore wind; positive = NE (upwelling-favourable), negative = SW (cold fronts)
SST	°C	Sea surface temperature
Rain	mm d^{-1}	Daily rainfall in the catchment
Discharge	m^3 s^{-1}	River discharge
PAR	mol m^{-2} d^{-1}	Daily photosynthetically active radiation
Tide_range	m	Daily tidal range

Files: `ex3_chla_numeric.csv` (Numeric-Attribute, Date as ID) and `ex3_chla_factors.csv` (Factor-Attribute: Year, Month 1–12, Season, Days).

The three model levels

Level	Predictors (day t)	No.	What it represents
M0	Chla	1	Chlorophyll's own memory: a machine-learning version of persistence
M1	The 7 raw variables	7	Today's environment
M2	M1 + 7-day mean of each variable (days t−7 … t−1)	14	The environment of the past week: accumulated wind, rain, discharge and light

Each level is trained twice: target **Chla_lead3** (Chla 3 days ahead) and **Chla_lead7** (7 days ahead) — **six models** in total, all with the same rows, the same partition and the same settings, so that the only difference between them is the information they receive.

> **Rule zero — no peeking into the future**
>
> Only the **target** is moved forward in time (lead). The predictors are never moved forward. A predictor for day *t* may only use information available on day *t* or earlier.

Step 1 — Load and look at the series

1. Create the Datalist `chla` with `ex3_chla_numeric.csv` as Numeric-Attribute and `ex3_chla_factors.csv` as Factor-Attribute.
2. Plot Chla, Wind_along and Discharge through time (Descriptive tools).
3. Correlation plot of the raw variables. Does same-day wind correlate with same-day Chla?

Step 2 — Create the targets with lead

1. Open the **Derived features** tool on the Numeric-Attribute of `chla`.
2. Select **Chla**, function **lead**, with k = **3** and k = **7**. Save the result as a new Datalist `chla_targets` (columns Chla_lead3 and Chla_lead7).
3. Check the alignment: Chla_lead7 on 2022-11-17 must equal Chla on 2022-11-24 (12.01).

> **Tip**
>
> lead creates NAs at the **end** of the series: the last 3 days have no Chla_lead3, and the last 7 days have no Chla_lead7. They are future days that do not exist in the data.

Step 3 — Create the 7-day means (level M2)

1. Open **Derived features** again on `chla`.
2. Select the 7 raw variables, function **rolling mean**, window **7**, and keep the original columns. Save as `chla_roll7` (14 columns).
3. Check one value: the 7-day mean for a day *t* is the mean of days *t−7* to *t−1*. Today's value is already in the raw column.

> **Tip**
>
> The rolling mean has no value on the first day of the series (there is no earlier day to average).

Step 4 — Create the M0 Datalist

Create a copy of `chla` that keeps only the column **Chla**, and name it `chla_M0`. Level M1 uses `chla` itself.

Step 5 — Remove NA rows and align all Datalists

1. **Filter observations → Na.omit** on `chla_targets` (removes the last 7 days) and on `chla_roll7` (removes 1 January 2020).
2. **Match IDs** so that `chla_M0`, `chla`, `chla_roll7` and `chla_targets` contain exactly the same days.

> **Checkpoint**
>
> - All four Datalists: **1819 rows**, from 2020-01-02 to 2024-12-24, no NA, Factor-Attribute (Year) present.

> **Tip**
> Remove these rows instead of imputing them. The NAs are not missing measurements: they are days for which the past (start) or the future (end) does not exist. Imputing them would invent data.

Step 6 — Set up training (the same for all six models)

Setting	Value	Why
Algorithm	Random Forest (ntree = 500)	
Partition	Factor **Year**, test level **2024**	Training 2020–2023 (1460 days); 2024 (359 days) is kept for the final test
Resampling	**Temporal block CV** by **Year** (Create New Temporal CV scheme, window Expanding)	Expanding window: train 2020 → validate 2021; 2020–21 → 2022; 2020–22 → 2023
Tuning	Grid search, tuneLength = 5	Selects mtry
Seed	Fix it (e.g. 1)	Everybody gets comparable results

> **Tip**
> Neighbouring days are almost identical. With a random partition, every test day is surrounded by training days, and the model looks better than it really is (temporal leakage). Validation must always move forward in time.

Step 7 — Train the six models

1. Train **M0** (X = `chla_M0`, Y = `chla_targets`, response **Chla_lead7**). Write down the CV **Rsquared** and RMSE for the best mtry.
2. Predict the test partition (2024) and write down R^2 and RMSE.
3. Repeat with **M1** (X = `chla`) and **M2** (X = `chla_roll7`).
4. Repeat the three models with response **Chla_lead3**.
5. Save a savepoint.

Step 8 — The persistence baseline

Persistence says "Chla in *h* days = Chla today". Its skill is the squared correlation between Chla and the target. In the correlation plot of `chla` together with `chla_targets` (2020–2023 rows), square the r of Chla with Chla_lead3 and with Chla_lead7.

Model	CV t+3	CV t+7	Test 2024 t+3	Test 2024 t+7
Persistence				
M0 — Chla only				
M1 — 7 raw variables				
M2 — raw + 7-day means				

Step 9 — Inspect the best model

1. Variable importance of M2 at t+7: which variables lead? Do the 7-day means of wind and discharge rank high?
2. Plot observed versus predicted through 2024. When does the model fail?

Think about it

- Does M0 beat persistence? Why can a random forest that only sees Chla be **worse** than repeating today's value?
- At t+3, which levels beat persistence? And at t+7? Is a model that does not beat persistence useful?
- Where do the 7-day means help more, at 3 or at 7 days? Does this match the 3–10-day response to wind described in the Context?
- Why is skill lower at 7 days than at 3 days?
- Is the 2024 test R^2 equal to the CV R^2? Which number would you report in a paper, and why?
- Did Tide_range become important in any model? Why would that be suspicious?

Going further (optional)

If you have time, add one more family of features with **Derived features** and see whether M2 improves:

- 3-day means (short upwelling pulses)
- lags of Wind_along and Discharge (1–10 days)
- sin and cos of the month or day-of-year (seasonality)
- anomalies: SST minus its 30-day mean

Add one family at a time, keep the same rows, partition and seed, and record what each family adds.

Applying the workflow to your own data

Before you start, write down in one sentence the scientific question your data should answer, and decide which of the three examples it resembles most.

Decision	Ask yourself
Task	Am I looking for groups (Example 1), predicting a value or class in space (Example 2), or forecasting in time (Example 3)?
Unit of observation	What is one row? Are rows independent, or grouped in space, time, sites or individuals?
Target	Numeric (regression) or factor (classification)? Is one class rare?
Predictors	Which are causal or mechanistic, which are proxies, and which could leak the answer?
Pre-processing	Missing values (how and why missing?), skewness, scaling, redundant variables
Validation	Which split reflects how the model will be used: new sites, new years, new regions?
Baseline	What simple rule must the model beat?

Reporting checklist

- State the question, the unit of observation and the target.
- Describe every pre-processing step, in order (imputation, transformation, scaling, variables removed).
- Report the validation design and why it matches the intended use.
- Report metrics on independent test data, together with a baseline.
- Report algorithm, tuning parameters and seeds; keep the iMESc savepoint as a reproducible record.
- Interpret variable importance cautiously: correlated predictors share or swap importance.

Glossary of metrics

Metric	Used for	Meaning
R^2	Regression	Share of the variance in the observed values explained by the predictions (1 = perfect; can be negative on test data)
RMSE / MAE	Regression	Typical prediction error, in the units of the target
Accuracy	Classification	Proportion of correctly classified observations; misleading when classes are imbalanced
Kappa	Classification	Agreement corrected for chance (0 = no better than chance, 1 = perfect)
Recall (per class)	Classification	Proportion of observations of a class that were correctly predicted
Quantization error	SOM	Mean distance between observations and their best-matching neuron
Topographic error	SOM	Proportion of observations whose first and second best-matching neurons are not neighbours
Persistence baseline	Forecasting	Forecast that the future equals the present value